Tell me how do an rule based code check a canvas for our color theory auction game

A rule-based code checker for a color theory auction game needs to evaluate player-submitted canvas artwork against specific auction criteria (such as color harmony, palette constraints, contrast ratios, or dominant moods). Since you're working with a canvas-based game, the checker typically works by extracting pixel data from the HTML5 `<canvas>` element and analyzing its color composition using standard color spaces (like RGB, HSL, or HSV).

Here is a breakdown of how you can build a robust, rule-based canvas validation engine for your color theory auction game:

1. Extracting the Canvas Data

To analyze any drawing, your code needs access to the raw pixel array using the canvas 2D rendering context method `getImageData`:

JavaScript

```
function analyzeCanvas(canvas) {
    const ctx = canvas.getContext('2d');
    const width = canvas.width;
    const height = canvas.height;

    // Get RGBA pixel values for the entire canvas
    const imageData = ctx.getImageData(0, 0, width, height);
    const pixels = imageData.data; // Uint8ClampedArray of [R, G, B, A, R, G, B
    // Pass to your rule engine
    return evaluateRules(pixels, width, height);
}
```

2. Converting RGB to HSL / HSV

Color theory rules (like complementary, analogous, or monochromatic schemes) are much easier to check in **HSL** (Hue, Saturation, Lightness) or **HSV** space than in raw RGB, because Hue directly represents the color wheel angle (0° to 360°).

Hue (H): The color family (e.g., Red = 0°, Green = 120°, Blue = 240°).

Saturation (S): Vividness or purity of the color.

Lightness/Value (L/V): How bright or dark the color is.

JavaScript

```javascript
function rgbToHsl(r, g, b) {
    r /= 255; g /= 255; b /= 255;
    let max = Math.max(r, g, b), min = Math.min(r, g, b);
    let h, s, l = (max + min) / 2;

    if (max === min) {
        h = s = 0; // Achromatic (grayscale)
    } else {
        let d = max - min;
        s = l > 0.5 ? d / (2 - max - min) : d / (max + min);
        switch (max) {
            case r: h = (g - b) / d + (g < b ? 6 : 0); break;
            case g: h = (b - r) / d + 2; break;
            case b: h = (r - g) / d + 4; break;
        }
        h /= 6;
    }
    return [h * 360, s * 100, l * 100];
}
```

3. Implementing Rule-Based Validators

Depending on the specific "auction card" or prompt the player received (e.g., *"Must use a complementary palette with high contrast"*), you can run specific checks on the sampled pixels.

A. Dominant Color Extraction (Color Palette Check)

To check if the player adhered to a specific palette (or avoided forbidden colors), you can build a color frequency map or use a simplified color quantization / binning approach:

JavaScript

```javascript
function getDominantHues(pixels, sampleStep = 16) {
    let hueBuckets = {}; // Group hues into 12 major color bins (30 degrees eac

    for (let i = 0; i < pixels.length; i += 4 * sampleStep) {
        let r = pixels[i], g = pixels[i+1], b = pixels[i+2], a = pixels[i+3];

        // Skip transparent or near-white background pixels if needed
        if (a < 128 || (r > 240 && g > 240 && b > 240)) continue;

        let [h, s, l] = rgbToHsl(r, g, b);

        // Ignore very low saturation (grays/whites) for hue checks
        if (s < 15 || l < 10 || l > 90) continue;

        let bucket = Math.floor(h / 30) * 30; // 12 buckets across 360 degrees
        hueBuckets[bucket] = (hueBuckets[bucket] || 0) + 1;
    }

    // Sort buckets by frequency
    return Object.entries(hueBuckets).sort((a, b) => b[1] - a[1]);
}
```

B. Checking Color Schemes (e.g., Complementary Rule)

If an auction rule requires a **Complementary Color Scheme** (two colors roughly 180° apart on the color wheel), your validator can check if the top two dominant hues satisfy the rule within an acceptable tolerance window (e.g., $\pm 15^\circ$):

JavaScript

```javascript
function checkComplementaryRule(pixels) {
    let sortedHues = getDominantHues(pixels);
    if (sortedHues.length < 2) return { pass: false, reason: "Not enough distir

    let primaryHue = parseFloat(sortedHues[0][0]);
    let secondaryHue = parseFloat(sortedHues[1][0]);

    // Calculate shortest angular distance on a 360-degree wheel
    let diff = Math.abs(primaryHue - secondaryHue);
    let distance = Math.min(diff, 360 - diff);
```

```
    // Complementary angle is 180 degrees (allow 20-degree tolerance)
    let isComplementary = Math.abs(distance - 180) <= 20;

    return {
        pass: isComplementary,
        primary: primaryHue,
        secondary: secondaryHue,
        distance: distance,
        reason: isComplementary ? "Valid complementary scheme!" : `Colors are s
    };
}
```

C. Contrast & Mood Checks (Value / Lightness)

If an auction card calls for a **High Contrast** or **Moody/Dark** theme, you can evaluate the Lightness (L) distribution across the canvas:

JavaScript

```
function checkContrastRule(pixels, sampleStep = 16) {
    let minL = 100, maxL = 0;

    for (let i = 0; i < pixels.length; i += 4 * sampleStep) {
        let r = pixels[i], g = pixels[i+1], b = pixels[i+2], a = pixels[i+3];
        if (a < 128) continue;

        let [, , l] = rgbToHsl(r, g, b);
        if (l < minL) minL = l;
        if (l > maxL) maxL = l;
    }

    let contrastRange = maxL - minL;
    let passHighContrast = contrastRange >= 60; // Requires deep shadows and bı

    return {
        pass: passHighContrast,
        contrastRange: contrastRange,
        reason: passHighContrast ? "Strong tonal contrast detected." : "Canvas
    };
}
```

4. Structuring the Auction Evaluation Pipeline

In your game loop, when the auction timer ends or the player submits their painting, pipe the canvas through a rule evaluator matching the current auction constraints:

JavaScript

```
function evaluateAuctionSubmission(canvas, auctionCriteria) {
    let results = { score: 0, feedback: [] };

    switch (auctionCriteria.type) {
        case 'COMPLEMENTARY':
            let compCheck = checkComplementaryRule(canvas.getContext('2d').getI
            results.pass = compCheck.pass;
            results.feedback.push(compCheck.reason);
            break;

        case 'HIGH_CONTRAST':
            let contrastCheck = checkContrastRule(canvas.getContext('2d').getIn
            results.pass = contrastCheck.pass;
            results.feedback.push(contrastCheck.reason);
            break;

        // Add more rules (Monochromatic, Analogous, Triadic, etc.)
    }

    // Calculate auction valuation score based on rule adherence and canvas pi
    results.valuation = results.pass ? Math.floor(Math.random() * 500 + 800) :
    return results;
}
```

Tips for Optimization

Pixel Sampling: Avoid checking every single pixel on large canvases (e.g., 800×600 has $480,000$ pixels). Use a step interval (`i += 16` or `32`) to sample every few pixels; it's virtually indistinguishable for color frequency analysis and runs instantly.

Background Masking: If your canvas has a solid white or black default background, make sure your algorithm filters out pixels matching the background color so they don't skew the dominant hue calculation.

Would you like help implementing a specific color harmony rule (like Triadic or Analogous) or setting up the scoring multiplier for the auction house valuation?

Like I want to tell the claude to implement these things how would I do it

To have an AI assistant like Claude implement this for your project, you want to provide a well-structured prompt that gives it clear context, technical constraints, and exact functional requirements.

Here is a prompt template you can copy, paste, and adapt for Claude to integrate this rule-based canvas checker directly into your codebase.

Copy-Paste Prompt for Claude

Markdown

```
I am building a web-based color theory auction game using HTML5 Canvas and Java

Please help me implement a robust evaluation module in JavaScript.

### Requirements:
1. **Pixel Data Extraction:** A function to extract RGBA pixel data from an HTM
2. **Color Space Conversion:** An efficient RGB-to-HSL conversion utility so we
3. **Smart Sampling & Filtering:** A way to sample pixels efficiently (skipping
4. **Specific Rule Validators:**
   - **Complementary Rule:** Checks if the top two dominant hues on the canvas
   - **High Contrast Rule:** Evaluates the Lightness ($L$) range across the sam
5. **Auction Valuation Engine:** A wrapper function that takes the canvas and a

### Codebase Context:
- My project uses vanilla JavaScript / HTML5 Canvas [or mention if you are usir
- Please write clean, modular ES6 JavaScript functions with helpful comments ex
```

Tips for Getting the Best Result from Claude:

Specify your framework: If your project uses React, Vue, or vanilla JS modules, mention it to Claude so it formats the code (e.g., as custom hooks, component methods, or utility classes) to fit your architecture.

Ask for edge cases: If players might submit blank canvases or scribble only one tiny dot, you can ask Claude to add safeguard checks (e.g., *"Make sure the code handles empty canvases or minimal paint gracefully without throwing errors"*).